

TelemetriaMqtt v2 — Checkpoint 7

Dashboard Grafana — caminho ao vivo

Projeto de Telemetria FSAE

18 de setembro de 2026

Abstract

Este documento detalha o sétimo checkpoint do projeto: o painel *ao vivo* do Grafana, que mostra a telemetria assim que ela chega ao broker MQTT, sem passar pelo InfluxDB. Usa o plugin `grafana-mqtt-datasource` e o Grafana Live, provisionados inteiramente como código. O documento também registra o que foi descoberto na prática sobre o plugin (inclusive uma armadilha que custou uma investigação) e, com a mesma clareza, o que **não** foi validado automaticamente.

Contents

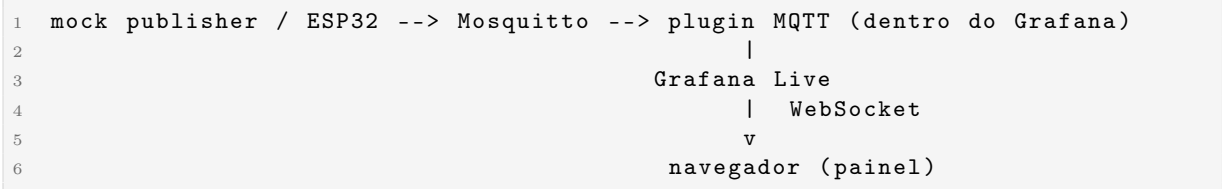
1	Contexto e objetivo	1
2	O que foi descoberto antes de escrever os testes	1
2.1	A armadilha do prefixo do canal	2
3	Estrutura adicionada	2
3.1	Plugin e senha no docker-compose	2
3.2	Datasource	3
3.3	Dashboard gerado por script	3
4	Testes	4
4.1	Resultado da execução	4
4.2	Como o TDD correu de fato	5
4.3	Um teste antigo tornou-se obsoleto	5
5	O que NÃO foi validado automaticamente	5
6	Validação manual (roteiro)	5
7	Pontos de atenção	6
8	Nota sobre metodologia	6
9	Resumo do que ficou pronto	6
10	Próximos passos	6

1 Contexto e objetivo

A arquitetura (SCOPE.md) tem dois caminhos deliberadamente separados. O **histórico** (checkpoints 4 a 6) passa por ingestão e InfluxDB e serve para análise pós-treino. O **ao vivo** (este

checkpoint) serve o pit/box: o Grafana assina o tópico MQTT diretamente e empurra os valores para o navegador, sem polling. Isso elimina o limite de refresh de cerca de 5 segundos de um painel que consulta banco, e faz o painel ao vivo continuar funcionando mesmo com o InfluxDB fora do ar.

Listing 1: Caminho ao vivo (trecho da arquitetura)



O plugin é, na prática, um cliente MQTT que roda *dentro* do Grafana: conecta no broker com o usuário `telemetry_reader` (somente leitura, pela ACL do checkpoint 3), assina o tópico `telemetry/esp32/data` e converte cada mensagem JSON em linhas de um *data frame*.

2 O que foi descoberto antes de escrever os testes

Antes dos testes, foi feito um **experimento descartável** (*spike*): um Grafana e um Mosquitto temporários, montados à mão, só para descobrir como o plugin realmente se comporta. Nada desse experimento foi versionado; os containers foram removidos e o que importa virou os testes e a implementação. Ele foi necessário porque não dá para escrever um teste correto de algo cujo contrato ainda se desconhece: o formato do nome do canal Live, por exemplo, não está documentado de forma que se possa testar sem um navegador.

Achado	Detalhe
Compatibilidade Instalação	Plugin 1.3.7, assinatura <code>valid</code> , exige Grafana ≥ 11 (usamos o 13.0.2). <code>GF_INSTALL_PLUGINS</code> está depreciado no Grafana 13; o recomendado é <code>GF_PLUGINS_PREINSTALL_SYNC</code> , que bloqueia a subida até instalar.
Configuração Query	<code>jsonData.uri</code> , <code>jsonData.username</code> e <code>secureJsonData.password</code> . Só tem o campo <code>topic</code> , enviado em <i>base64 URL-safe</i> sem padding (restrição de caracteres dos canais Live).
Nome do canal	<code>ds/<uid>/<intervalo>/<tópico em base64>/<streamingKey></code> , devolvido pela query em <code>meta.channel</code> .
Formato do frame	Uma coluna <code>Time</code> mais uma coluna por chave do JSON (<code>ts</code> , <code>rpm</code> , ...), todas numéricas.
Agrupamento	O plugin junta as mensagens recebidas e só empurra um frame a cada <i>intervalo</i> ; esse intervalo é o piso da latência.
Health check	<code>POST /api/datasources/uid/<uid>/health</code> abre uma conexão MQTT de verdade e devolve <code>status OK</code> ou <code>ERROR</code> com a causa.

2.1 A armadilha do prefixo do canal

A primeira tentativa de assinar o canal pelo WebSocket falhou com `namespace does not reference a valid org ID`. Prefixando o canal com `1/` (o ID da organização), a assinatura foi **aceita sem erro** (`"subscribe": {}`), mas nenhum dado chegava, nem mesmo o stream de teste nativo do Grafana. O log de debug mostrou o motivo: o Grafana 13 publica os frames em `default/ds/...`, e o `default` é o namespace da organização 1. Assinar `1/ds/...` entra num canal diferente, que fica mudo para sempre e sem nenhum erro. Com o prefixo `default/`, os dados passaram a chegar imediatamente. O mesmo cliente serviu para o stream nativo e para o MQTT, o que descartou a hipótese de problema no plugin.

3 Estrutura adicionada

Listing 2: Arquivos novos e alterados

```
1 infra/
2   docker-compose.yml                (grafana: + plugin e
   MQTT_READER_PASSWORD)
3   .env.example                      (+ MQTT_READER_PASSWORD)
4   requirements-test.txt              (+ websockets)
5   grafana/
6     generate_live_dashboard.py       (novo)
7     dashboards/ao-vivo.json          (novo, gerado)
8     provisioning/datasources/mqtt.yml (novo)
9   tests/
10    test_grafana_live.py              (novo, 6 testes)
11    conftest.py                      (fixture de .env aprende a variá
   vel nova)
12    test_docker_compose.py            (1 linha: ver seção de testes)
```

3.1 Plugin e senha no docker-compose

Listing 3: Trecho novo do serviço grafana

```
1 GF_PLUGINS_PREINSTALL_SYNC: grafana-mqtt-datasource@1.3.7
2 MQTT_READER_PASSWORD: ${MQTT_READER_PASSWORD:?defina no infra/.env ...}
```

A versão do plugin é fixada de propósito, para a stack ser reproduzível. O sufixo `_SYNC` é essencial: sem ele o Grafana poderia tentar provisionar o datasource antes de o plugin estar instalado. A senha segue o mesmo padrão do InfluxDB: obrigatória (`?:`) e lida do `infra/.env`, que não é versionado. **Atenção operacional:** essa senha precisa ser a mesma cadastrada no `passwd` do Mosquitto para `telemetria_reader`. São dois lugares que o projeto não sincroniza sozinho, por design (nenhuma credencial em arquivo versionado).

3.2 Datasource

Listing 4: `infra/grafana/provisioning/datasources/mqtt.yml`

```
1 datasources:
2   - name: MQTT
3     uid: mqtt-live
4     type: grafana-mqtt-datasource
5     access: proxy
6     isDefault: false
7     jsonData:
8       uri: tcp://mosquitto:1883
9       username: telemetria_reader
10    secureJsonData:
11      password: $MQTT_READER_PASSWORD
```

`mosquitto` é o nome do serviço na rede interna do compose. `isDefault: false` mantém o InfluxDB como datasource padrão. Sem TLS em dev local; na VPS (checkpoint 10) a URI passa a `tls://`.

3.3 Dashboard gerado por script

O dashboard `telemetria-ao-vivo` é gerado por `generate_live_dashboard.py`, que **importa a lista de painéis do dashboard histórico**. Os dois dashboards têm sempre os mesmos 7 agrupamentos e unidades, com uma única fonte de verdade. Como o plugin devolve um frame

único com todos os campos, cada painel usa uma transformação para ficar só com os do seu grupo:

Listing 5: Painel ao vivo (JSON gerado, simplificado)

```
1 {
2   "type": "timeseries",
3   "title": "Temperaturas (°C)",
4   "datasource": {"type": "grafana-mqtt-datasource", "uid": "mqtt-live"},
5   "interval": "200ms",
6   "transformations": [{
7     "id": "filterFieldsByName",
8     "options": {"include": {"names": ["Time", "engineTemp", "airTemp"]}}
9   ]},
10  "targets": [{"refId": "A", "topic": "telemetria/esp32/data", ...}]
11 }
```

Latência. O campo `interval` (o “Min interval” do painel) é o intervalo de agrupamento do plugin. Foi fixado em 200 ms: 5 atualizações por segundo, cerca de 4 amostras por frame com o dispositivo a 20 Hz. Isso cabe no “centenas de ms” do escopo sem sobrecarregar o navegador. Por precaução (**não medido**), a janela de tempo é fixa em 1 minuto, com o seletor de tempo escondido: o Grafana também deriva o intervalo da janela e da largura do painel, e uma janela muito maior tenderia a espaçar os frames. O plugin não guarda histórico; para isso existe o dashboard histórico.

4 Testes

Seis testes novos (`infra/tests/test_grafana_live.py`), todos contra o **Grafana real do docker-compose.yml**. O Mosquitto sobe com o `mosquitto.conf` e o `acl.conf` reais, mas com um `passwd` descartável criado num diretório temporário e injetado por um arquivo de *override* do compose. Assim os testes não dependem nem tocam no seu `passwd` e no seu `.env` reais. Se a stack já estava de pé, ela é restaurada ao final; o que estava parado volta a ficar parado.

- Teste 1** `test_live_dashboard_generator_output_matches_committed_file` — o JSON versionado é idêntico ao que o gerador produz hoje; evita alguém editar um dos dois à mão.
- Teste 2** `test_mqtt_plugin_is_installed_with_valid_signature` — o plugin está instalado, é do tipo `datasource` e tem assinatura válida (um plugin não assinado seria recusado).
- Teste 3** `test_mqtt_datasource_is_provisioned` — o `datasource` existe com tipo, URI e usuário corretos, a senha está configurada (sem expô-la), e o InfluxDB continua sendo o padrão.
- Teste 4** `test_mqtt_datasource_health_check_authenticates_against_real_broker` — o `health check` do plugin autentica no Mosquitto de verdade; valida rede entre containers, `passwd` e a variável de senha numa chamada só.
- Teste 5** `test_live_dashboard_is_provisioned_with_one_streaming_panel_per_group` — o dashboard aparece com 7 painéis do tipo série temporal, todos no `datasource MQTT` e no tópico do `publisher`, com intervalo ≤ 500 ms. A união dos campos filtrados é **exatamente** os 20 sinais do `mock_publisher`.
- Teste 6** `test_mock_publisher_message_reaches_grafana_live_websocket` — o mais forte do checkpoint. O código de produção do `mock publisher` publica no broker real; o plugin real, dentro do Grafana real, recebe; e um cliente `WebSocket` (o mesmo caminho que o navegador usa) assina o canal e recebe os valores. Confere que chegam todos os 20 sinais mais `Time` e `ts`, com RPM entre 800 e 8000 e `ts` entre os valores realmente publicados. O

teste publica em loop a 20 Hz, como o dispositivo real, em vez de uma vez só, porque o plugin só assina o tópico no broker depois que o stream começa.

4.1 Resultado da execução

Listing 6: Saída real de pytest -v (arquivo novo)

```
1 test_grafana_live.py::
  test_live_dashboard_generator_output_matches_committed_file PASSED
2 test_grafana_live.py::test_mqtt_plugin_is_installed_with_valid_signature
  PASSED
3 test_grafana_live.py::test_mqtt_datasource_is_provisioned PASSED
4 test_grafana_live.py::
  test_mqtt_datasource_health_check_authenticates_against_real_broker
  PASSED
5 test_grafana_live.py::
  test_live_dashboard_is_provisioned_with_one_streaming_panel_per_group
  PASSED
6 test_grafana_live.py::
  test_mock_publisher_message_reaches_grafana_live_websocket PASSED
7 6 passed in 5.29s
```

Suítes completas ao final do checkpoint: infra **29** testes, backend **43**, firmware **1**, todos passando, sem nenhum container sobrando depois da execução.

4.2 Como o TDD correu de fato

1. **Vermelho:** os 6 testes foram escritos antes de qualquer implementação e rodados. Os 6 falharam, cada um pelo motivo certo (gerador inexistente, plugin não instalado, datasource ausente), e não por erro de fixture.
2. **Verde:** implementação do compose, datasource e gerador. Cinco testes passaram de imediato, incluindo o principal.
3. **Um erro do meu próprio teste:** o teste 3 lia `secureJsonFields` da listagem `/api/datasources`, que não traz esse campo (só o endpoint de um datasource individual traz). Confirmei isso na API antes de mexer e apenas passei a ler do endpoint certo, com a asserção idêntica. É correção de uma premissa errada sobre a API, num teste que ainda nunca tinha passado, e não mudança de requisito.

4.3 Um teste antigo tornou-se obsoleto

A suíte completa revelou que `test_docker_compose_up_starts_a_working_grafana` (checkpoint 6) afirmava `len(datasources) == 1`. Esse requisito realmente mudou: o checkpoint 7 adiciona legitimamente um segundo datasource. Pela regra do projeto (testes existentes são intocáveis), **parei e perguntei**, sem alterar o teste por conta própria. A decisão foi a mudança mínima: `== 1` passou a `== 2`, e nada mais naquele teste foi alterado.

5 O que NÃO foi validado automaticamente

Os testes provam que o backend está correto de ponta a ponta: o dado sai do mock, passa pelo broker e pelo plugin, e chega ao cliente WebSocket do Grafana Live. Eles **não** provam o que acontece dentro do navegador: se os gráficos efetivamente desenharam, e principalmente se a transformação `filterFieldsByName` funciona sobre dados em *streaming*. Isso não foi verificado, por não haver navegador automatizado neste ambiente. É o ponto mais provável de surpresa e exige a validação manual abaixo.

6 Validação manual (roteiro)

1. Coloque a senha real do `telemetry_reader` em `MQTT_READER_PASSWORD` no `infra/.env`. (Se o seu `.env` é anterior a este checkpoint, os testes já acrescentaram a variável com um valor de teste; troque pelo real.)
2. `cd infra && docker compose up -d`. A primeira subida baixa o plugin, então precisa de internet.
3. Em outro terminal, com o `venv` do backend ativo: `python -m mock_publisher --no-tls --port 1883 --username esp32_telemetria --password <senha>`. O `--no-tls` e a porta 1883 são necessários porque o publisher assume por padrão TLS na porta 8883, e o broker local do `compose` não usa TLS.
4. Abra `http://localhost:3000/d/telemetry-ao-vivo` e confirme que os 7 painéis desenham curvas em movimento, com o RPM oscilando entre 800 e 8000.
5. Se um painel ficar vazio, a causa mais provável é a transformação não se aplicar ao streaming (hipótese, não confirmada). Nesse caso, a alternativa é esconder os campos por *overrides* em vez de filtrar por nome.

7 Pontos de atenção

- **Duas fontes para a mesma senha:** `.env` e `passwd` do Mosquitto precisam coincidir, senão o painel ao vivo fica sem dados (o health check do datasource mostra o erro de autenticação).
- **Dependência de rede na primeira subida:** o plugin é baixado do `grafana.com`. Depois fica no volume `grafana-data`.
- **Sessões MQTT órfãs** (lido no código do plugin, não medido): ele conecta com *clean session* desligado e client ID aleatório, então cada reinício do Grafana pode deixar uma sessão persistente no Mosquitto. O custo é desprezível (assinatura QoS 0, sem fila), mas cresce. Se virar problema: fixar o `clientId` no datasource ou definir `persistent_client_expiration` no broker.
- **Escopo de segurança:** o Grafana só lê. A ACL do `telemetry_reader` não permite publicar, e o plugin também recusa publicações pelo próprio Live.

8 Nota sobre metodologia

O checkpoint 6 registrou uma inversão de TDD (infraestrutura primeiro, testes depois) e prometeu que, a partir do 7, o teste voltaria a vir primeiro. Foi o que aconteceu: vermelho, depois verde, como descrito acima. A única ressalva honesta é o experimento descartável descrito no início. Ele veio antes dos testes, mas serviu para descobrir o contrato do plugin, não para construir a implementação, e nenhum arquivo dele foi versionado.

Por decisão de metodologia, toda escolha de tecnologia foi registrada no `SCOPE.md` na seção “Stack tecnológico”: o plugin `grafana-mqtt-datasource` 1.3.7 e a biblioteca `websockets` dos testes. O `CLAUDE.md` ganhou o comando de regeneração do dashboard ao vivo e a nota operacional da nova variável.

9 Resumo do que ficou pronto

- Plugin MQTT do Grafana instalado por versão fixada, com o datasource provisionado como código e credenciais só por variável de ambiente.
- Dashboard ao vivo de 7 painéis, gerado por script a partir da mesma lista do histórico, atualizando a cada 200 ms.
- Prova automatizada, contra Grafana e Mosquitto reais, de que a mensagem publicada chega ao Grafana Live pelo mesmo caminho que o navegador usa.
- Contrato do canal Live documentado, incluindo a armadilha do prefixo `1/` versus `default/`.

10 Próximos passos

Este checkpoint espera a sua revisão. Se aprovado, o commit fecha o checkpoint com a tag `v0.7.0`. O próximo é o **checkpoint 8**: firmware do ESP32, com o parser CAN (TWAI) implementando a tabela de payloads do `SCOPE.md`, começando pelos testes nativos com Unity.